

Matrix Theory and Computation

Part 2: Structure, fast transforms, and efficient computation

Original slides 30–87 | Detailed lecture notes

Detailed teaching and study notes

First-year postgraduate course

Lecturer: Huiwen Wu

Step-by-step calculations, algorithm derivations, teaching prompts,
verified textbook exercises, and worked solutions.

Based on the supplied Chapter 1 slides and Golub and Van Loan,
Matrix Computations, 4th edition.

Contents

Teaching guide and learning goals	2
1. Banded matrices and a discrete differential operator	3
2. Triangular products: derive the legal index range	5
3. Band storage: diagonal offsets become row indices	6
4. Diagonal, symmetric, and permutation matrices	8
5. Block matrices and reusable subproblems	10
6. Kronecker products and vectorization	11
7. Hamiltonian and symplectic structure	14
8. Strassen: reorganizing arithmetic	16
9. The DFT is a matrix; the FFT is an algorithm	17
10. Sine and cosine transforms: boundaries and normalization	20
11. Haar wavelets: analysis, inverse, and approximation	22
12. Data movement, stride, and blocking	25
13. Parallel work: ownership, balance, and communication	27
14. Cannon's algorithm: a complete four-processor trace	29
15. Textbook exercises: assignment	31
16. Textbook exercises: worked solutions	32
17. Additional exercises with solutions	34
18. A computational workshop and assessment guide	35
References and exercise provenance	37

Teaching guide and learning goals

Audience: first-year postgraduate students. These notes assume that students can multiply small matrices and understand a dot product, an outer product, and the assignment $y \leftarrow y + Ax$. Those ideas are briefly recalled when needed.

Split boundary. The accompanying Part 2 slide PDF contains original pages **30–87**. Page 30 begins the tridiagonal-stencil example; the band-matrix definition was on page 29. This document repeats that definition so the notes can be read independently. All slide references below use original page numbers; subtract 29 to obtain the local Part 2 page.

Main question: how does mathematical structure change what we compute, what we store, and what we communicate?

By the end, students should be able to implement a banded matrix-vector product, derive triangular summation bounds, perform block and Kronecker calculations, trace a four-point FFT and Haar transform, and explain why locality and communication influence performance.

A realistic teaching route

There are 58 slides in this part and several substantial topics. A 90-minute lesson can teach a selected core, with the remaining material used in later tutorials or independent study. The detailed notes cover **all** the supplied examples; they should not be read aloud in one sitting.

Time	Original slides	Teaching and student work
0–7 min	29 recap; 30–31	Bandwidth, stencil, and the boundary effect.
7–17 min	32–36	Triangular sums and explicit band packing.
17–25 min	37–40	Symmetric storage and permutation indices.
25–36 min	41–47	Block multiplication and Kronecker products.
36–45 min	48–49	Derive and verify the vec identity.
45–61 min	55–58	Four-point DFT and FFT butterflies.
61–70 min	59–61	Sine eigenvector; explain DCT normalization.
70–82 min	62–66	Haar analysis, inverse, and discarded energy.
82–87 min	69–72	One locality example and the cache budget.
87–90 min	84	Exit check and exercise assignment.

Suggested follow-up allocation. Give symplectic structure 15–20 minutes, Strassen 15–20 minutes, and memory/parallel computation 45–60 minutes in tutorials. Students benefit from actually tracing Cannon’s two rounds instead of seeing it only as a named method.

Teaching pattern. For every example: state dimensions and assumptions; predict which terms are needed; calculate a small instance; derive the algorithm; count the work; ask what would break if the assumptions changed.

1. Banded matrices and a discrete differential operator

Slides 29–31; E09. A band matrix has lower bandwidth at most p and upper bandwidth at most q when

$$a_{ij} = 0 \quad \text{if } i - j > p \text{ or } j - i > q.$$

Its nonzeros are restricted to a strip near the main diagonal. These are structural bounds: entries inside the strip may still be zero.

For a fixed row i , allowed columns satisfy

$$\max(1, i - p) \leq j \leq \min(n, i + q).$$

For a fixed column j , allowed rows satisfy

$$\max(1, j - q) \leq i \leq \min(m, j + p).$$

Write both inequalities on the board. They are equivalent descriptions, but the positions of p and q differ.

E09: compute the tridiagonal product

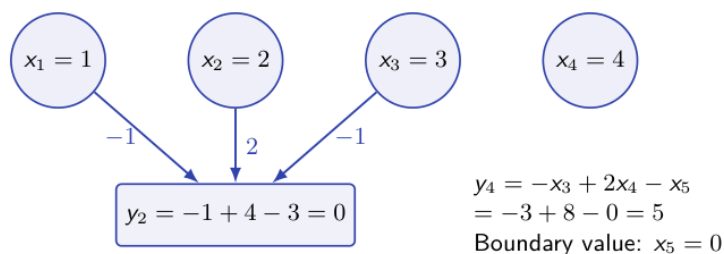
Let

$$T = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}.$$

Here $p = q = 1$. Compute each row:

$$\begin{aligned} (Tx)_1 &= 2(1) - 2 = 0, \\ (Tx)_2 &= -1 + 2(2) - 3 = 0, \\ (Tx)_3 &= -2 + 2(3) - 4 = 0, \\ (Tx)_4 &= -3 + 2(4) = 5. \end{aligned}$$

Thus $Tx = (0, 0, 0, 5)^T$.



The tridiagonal matrix couples each sample only to its immediate neighbours.

Figure 1: Original slide 31: a local stencil uses only neighboring samples.

Why a linear sequence gives a nonzero boundary output

Introduce two boundary values, $x_0 = x_5 = 0$. Then all rows use

$$(Tx)_i = -x_{i-1} + 2x_i - x_{i+1}.$$

At interior points, a linear sequence satisfies

$$-(i-1) + 2i - (i+1) = 0.$$

At the right boundary the matrix uses $x_5 = 0$, not the linear continuation $x_5 = 5$. Therefore the last output is 5. The nonzero entry is a consequence of the boundary condition, not an error in the interior formula.

On a uniform mesh with spacing h ,

$$-u''(x_i) \approx \frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2}.$$

The slide omits the factor h^{-2} . Multiplying by T applies an operator to a supplied vector; solving $Tu = f$ is a different task.

Added boundary comparison. If the right boundary were prescribed as $u_5 = 5$, the same interior vector $u = (1, 2, 3, 4)^T$ would have zero second differences throughout. In a linear system, a known boundary value moves into the right-hand side: the last equation is $-u_3 + 2u_4 = f_4 + u_5$.

Storage and arithmetic

A full tridiagonal pattern has

$$\text{nnz}(T) = n + 2(n-1) = 3n - 2.$$

For $n = 4$, the generic sparse update $y \leftarrow y + Tx$ costs $2(10) = 20$ flops, compared with $2(16) = 32$ for a dense update loop.

For a fresh product with direct sums, the count differs. There are $3n - 2$ multiplications and $2(n - 2) + 2 = 2n - 2$ additions for $n \geq 2$, totaling $5n - 4$. Also, the particular coefficients -1 can be implemented as subtractions. The slide's 20-flop comparison counts generic nonzero multiply-add updates, not the most specialized possible stencil code.

Matrix-free implementation

```
Given x[1:n], with n >= 2:
y[1] = 2*x[1] - x[2]
for i = 2,...,n-1:
    y[i] = -x[i-1] + 2*x[i] - x[i+1]
y[n] = -x[n-1] + 2*x[n]
```

For $n = 1$, handle $y_1 = 2x_1$ separately. This algorithm stores no matrix and uses $O(n)$ work. “Matrix-free” means that we know how to apply the linear map; it does not mean that the map lacks a matrix representation.

Check. For $x = (1, 1, 1, 1)^T$, the answer is $(1, 0, 0, 1)^T$, again reflecting zero boundary values.

2. Triangular products: derive the legal index range

Slides 32–33; E10. If U and V are upper triangular, then $u_{ik} = 0$ for $k < i$ and $v_{kj} = 0$ for $k > j$. Thus a nonzero contribution requires

$$i \leq k \leq j.$$

If $i > j$, no such index exists, so UV is upper triangular.

The valid formula is

$$(UV)_{ij} = \sum_{k=i}^j u_{ik}v_{kj}, \quad i \leq j.$$

E10: all six upper-triangular entries

Use

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Then

$$\begin{aligned} c_{11} &= 1(2) = 2, & c_{12} &= 1(1) + 2(3) = 7, \\ c_{13} &= 1(0) + 2(2) + 3(1) = 7, & c_{22} &= 4(3) = 12, \\ c_{23} &= 4(2) + 5(1) = 13, & c_{33} &= 6(1) = 6. \end{aligned}$$

Therefore

$$UV = \begin{bmatrix} 2 & 7 & 7 \\ 0 & 12 & 13 \\ 0 & 0 & 6 \end{bmatrix}.$$

The c_{13} calculation includes a numerical zero inside the allowed pattern. An upper-triangular algorithm exploits triangular structure, not every additional accidental zero.

Exact operation count

For $C \leftarrow C + UV$, the number of multiply-add updates is

$$N = \sum_{i=1}^n \sum_{j=i}^n (j - i + 1).$$

Set $d = j - i$. There are $n - d$ positions with that offset:

$$N = \sum_{d=0}^{n-1} (n - d)(d + 1) = \frac{n(n + 1)(n + 2)}{6}.$$

An alternative derivation counts triples $1 \leq i \leq k \leq j \leq n$. This is the number of length-three nondecreasing choices from n values.

The update work is

$$2N = \frac{n(n + 1)(n + 2)}{3} = \frac{n^3}{3} + O(n^2).$$

For $n = 3$, $N = 10$, hence 20 flops, versus 54 for a full dense update. The asymptotic ratio is $1/6$; the finite $n = 3$ ratio is $20/54$.

```

for i = 1,...,n:
    for j = i,...,n:
        for k = i,...,j:
            C[i,j] += U[i,k]*V[k,j]

```

Storage caution. If the lower triangle of a dense output array contains old data, this loop does not clear it. Initialize the output appropriately or use a data structure that records only the upper triangle.

Check. Upper times lower triangular is generally dense. The bound $i \leq k \leq j$ was derived specifically for two upper-triangular factors.

3. Band storage: diagonal offsets become row indices

Slides 34–36; E11. Store a banded $m \times n$ matrix in an array with $p + q + 1$ rows:

$$\text{AB}(i - j + q + 1, j) = a_{ij}.$$

The original column j is unchanged. The storage row records the diagonal offset. The main diagonal has $i = j$, so its storage row is $q + 1$.

E11: pack every diagonal

The example is

$$A = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 4 & 5 & 6 & 7 & 0 \\ 0 & 8 & 9 & 10 & 11 \\ 0 & 0 & 12 & 13 & 14 \\ 0 & 0 & 0 & 15 & 16 \end{bmatrix}, \quad p = 1, \quad q = 2.$$

Its compact array is

$$\text{AB} = \begin{bmatrix} * & * & 3 & 7 & 11 \\ * & 2 & 6 & 10 & 14 \\ 1 & 5 & 9 & 13 & 16 \\ 4 & 8 & 12 & 15 & * \end{bmatrix}.$$

The four rows correspond to offsets $j - i = 2, 1, 0, -1$. Stars are unused positions, not matrix entries.

For $a_{24} = 7$,

$$r = 2 - 4 + 2 + 1 = 1, \quad \text{AB}(1, 4) = 7.$$

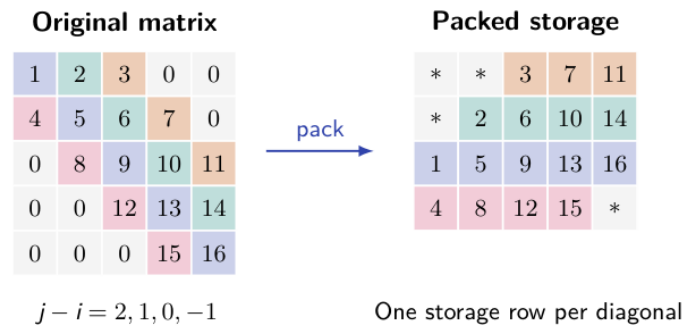
For $a_{43} = 12$,

$$r = 4 - 3 + 2 + 1 = 4, \quad \text{AB}(4, 3) = 12.$$

To invert the mapping, solve $r = i - j + q + 1$:

$$i = r + j - q - 1.$$

For example, storage position $(2, 4)$ corresponds to matrix row $i = 3$, hence $a_{34} = 10$.



Trace the value 12: a_{43} maps to $AB(4, 3)$ because $4 - 3 + 2 + 1 = 4$.

Figure 2: Original slide 36: diagonal offsets determine the storage rows.

Apply the packed matrix without unpacking

```
for j = 1,...,n:
    first = max(1,j-q)
    last  = min(m,j+p)
    for i = first,...,last:
        r = i-j+q+1
        y[i] += AB[r,j]*x[j]
```

If first exceeds last, the inner loop is empty. Every accessed position corresponds to a real matrix entry; stars are never read.

For $x = \mathbf{1}$, direct row sums give

$$Ax = (6, 22, 38, 39, 31)^T.$$

The packed-column trace is:

After column	Accumulated output, starting from zero
1	$(1, 4, 0, 0, 0)^T$
2	$(3, 9, 8, 0, 0)^T$
3	$(6, 15, 17, 12, 0)^T$
4	$(6, 22, 27, 25, 15)^T$
5	$(6, 22, 38, 39, 31)^T$

There are 16 retained entries and 4 padding positions. For an $n \times n$ full band pattern, $0 \leq p, q < n$,

$$N_{\text{band}} = n(p + q + 1) - \frac{p(p + 1)}{2} - \frac{q(q + 1)}{2}.$$

This counts allowed positions; additional zeros may lower the actual nonzero count. For fixed bandwidth, storage and application work grow linearly in n .

Library convention caution. The packed format above is for banded matrix-vector application. A factorization routine may require additional rows to store fill-in or pivoting information. Do not assume one array layout works for every routine.

4. Diagonal, symmetric, and permutation matrices

Slides 37–40; E12–E13. Different structures save different resources.

E12a: diagonal scaling

For $D = \text{diag}(2, -1, 3)$ and $x = (1, 2, -1)^T$,

$$Dx = (2, -2, -3)^T.$$

Store only the diagonal vector d . A fresh product costs n multiplications; the update $y \leftarrow y + Dx$ also needs n additions.

Left multiplication by a diagonal matrix scales rows; right multiplication scales columns. If

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad D = \text{diag}(2, 3),$$

then

$$DA = \begin{bmatrix} 2 & 4 \\ 9 & 12 \end{bmatrix}, \quad AD = \begin{bmatrix} 2 & 6 \\ 6 & 12 \end{bmatrix}.$$

The order matters even for a diagonal factor.

E12b: symmetric storage does not halve all arithmetic

Take

$$S = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 4 \\ 0 & 4 & 5 \end{bmatrix}, \quad z = (1, 2, 3)^T.$$

The diagonal contributes $(2, 6, 15)^T$. The stored lower entry $s_{21} = 1$ contributes $1z_1 = 1$ to output 2 and $1z_2 = 2$ to output 1. Entry $s_{32} = 4$ contributes $4z_2 = 8$ to output 3 and $4z_3 = 12$ to output 2. Hence

$$Sz = (2, 6, 15)^T + (2, 1, 0)^T + (0, 12, 8)^T = (4, 19, 23)^T.$$

```
for j = 1,...,n:
    y[j] += S[j,j]*z[j]
    for i = j+1,...,n:
        y[i] += S[i,j]*z[j]
        y[j] += S[i,j]*z[i]
```

Only $n(n+1)/2$ positions need storage. But each off-diagonal value influences two outputs. The generic update count is

$$2n + 4 \frac{n(n-1)}{2} = 2n^2.$$

For the example, six lower-triangle positions include one zero; exploiting that extra zero can save further work.

E13: a permutation is an index operation

Let

$$v = (3, 1, 2), \quad P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Then

$$Px = (x_3, x_1, x_2)^T.$$

For $x = (10, 20, 30)^T$, the result is $(30, 10, 20)^T$.

The inverse index vector is $(2, 3, 1)$: it retrieves the first original value from output position 2, the second from position 3, and the third from position 1. Thus $P^{-1} = P^T$ and $P^T Px = x$.

For a compatible matrix A , PA reorders rows. AP^T reorders columns using the same index list v , because

$$(AP^T)_{ij} = A_{i, v_j}.$$

Using AP generally gives a different order.

A gather implementation performs no floating-point multiplication. It still incurs integer indexing, loads, stores, and possibly irregular access. “Zero flops” does not mean “zero runtime.”

Check. Does applying a permutation twice undo it? Only when $P^2 = I$, such as a simple swap. The three-cycle above is undone after three applications.

5. Block matrices and reusable subproblems

Slides 41–44; E14. Blocking groups entries into submatrices. It changes organization, not the underlying linear map.

Suppose a row partition has sizes m_i , a shared partition has sizes r_k , and a column partition has sizes n_j . Then

$$A_{ik} \in \mathbb{R}^{m_i \times r_k}, \quad B_{kj} \in \mathbb{R}^{r_k \times n_j}.$$

Every term in

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

has size $m_i \times n_j$. Matching total matrix dimensions is insufficient if the chosen block boundaries disagree along the shared dimension.

Derivation from scalar multiplication

Fix a row in block i and column in block j . Partition the scalar sum over the shared index into its r_k -sized groups. The terms in group k are exactly the corresponding entry of $A_{ik} B_{kj}$. Summing all groups proves the block formula.

The block transpose satisfies $(A^T)_{ij} = A_{ji}^T$: exchange block positions and transpose each block.

E14: calculate the full product

With $I = I_2$,

$$A = \begin{bmatrix} I & 2I \\ 0 & I \end{bmatrix}, \quad B = \begin{bmatrix} 3I & I \\ I & 0 \end{bmatrix},$$

compute:

$$\begin{aligned} C_{11} &= I(3I) + (2I)I = 5I, \\ C_{12} &= I(I) + (2I)0 = I, \\ C_{21} &= 0(3I) + I(I) = I, \\ C_{22} &= 0(I) + I(0) = 0. \end{aligned}$$

Therefore

$$AB = \begin{bmatrix} 5 & 0 & 1 & 0 \\ 0 & 5 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

For $x = (1, 2, 3, 4)^T$, partition $x_1 = (1, 2)^T$, $x_2 = (3, 4)^T$. Then

$$Ax = \begin{bmatrix} x_1 + 2x_2 \\ x_2 \end{bmatrix} = \begin{bmatrix} 7 \\ 10 \\ 3 \\ 4 \end{bmatrix}.$$

Why block order cannot generally be reversed

The example uses multiples of the identity, which commute. General matrix blocks do not. For

$$R = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad D = \text{diag}(1, 2),$$

we have

$$RD = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix} \neq DR = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}.$$

Keep the factor order in every block formula.

Computational meaning. A block update $C_{ij} \leftarrow C_{ij} + A_{ik}B_{kj}$ can call a matrix-matrix kernel. With a suitable block size, repeatedly used data remain in fast memory. Blocking usually improves reuse without changing the leading dense flop count.

6. Kronecker products and vectorization

Slides 45–49; E15–E16. If B is $p \times q$ and C is $m \times n$, then $B \otimes C$ has size $pm \times qn$. Its block (i, j) is $b_{ij}C$.

E15: expand the blocks

Let

$$B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}.$$

Then

$$B \otimes C = \begin{bmatrix} C & 2C \\ 0 & C \end{bmatrix} = \begin{bmatrix} 2 & 0 & 4 & 0 \\ 0 & 3 & 0 & 6 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}.$$

The scalar $b_{12} = 2$ scales an entire block. The zero b_{21} creates a zero block.

B	$\otimes$	C	$=$	$B \otimes C$																								
<table border="1" style="border-collapse: collapse;"> <tr><td style="padding: 2px 10px;">1</td><td style="padding: 2px 10px; background-color: #f4a460;">2</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">1</td></tr> </table>	1	2	0	1		<table border="1" style="border-collapse: collapse;"> <tr><td style="padding: 2px 10px;">2</td><td style="padding: 2px 10px;">0</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">3</td></tr> </table>	2	0	0	3		<table border="1" style="border-collapse: collapse;"> <tr><td style="padding: 2px 10px;">2</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px; background-color: #f4a460;">4</td><td style="padding: 2px 10px;">0</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">3</td><td style="padding: 2px 10px; background-color: #f4a460;">0</td><td style="padding: 2px 10px; background-color: #f4a460;">6</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">2</td><td style="padding: 2px 10px;">0</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">3</td></tr> </table>	2	0	4	0	0	3	0	6	0	0	2	0	0	0	0	3
1	2																											
0	1																											
2	0																											
0	3																											
2	0	4	0																									
0	3	0	6																									
0	0	2	0																									
0	0	0	3																									

The scalar $b_{12} = 2$ becomes the entire upper-right block $2C$.

Try it: Which block must vanish when $b_{21} = 0$?

Figure 3: Original slide 47: each scalar of the first factor expands into a block.

The mixed-product identity is

$$(B \otimes C)(D \otimes F) = (BD) \otimes (CF)$$

when the factors are conformable. At block position (i, j) , the left side gives

$$\sum_k (b_{ik}C)(d_{kj}F) = \left(\sum_k b_{ik}d_{kj} \right) CF.$$

This is the corresponding block of the right side.

Transposition and inversion follow:

$$(B \otimes C)^T = B^T \otimes C^T, \quad (B \otimes C)^{-1} = B^{-1} \otimes C^{-1},$$

with the inverse identity requiring square nonsingular factors.

Column stacking and the vec identity

For $X \in \mathbb{R}^{n \times q}$, stack its columns:

$$\text{vec}(X) = \begin{bmatrix} X(:, 1) \\ \vdots \\ X(:, q) \end{bmatrix}.$$

If $C \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$, then

$$Y = CXB^T \in \mathbb{R}^{m \times p}$$

and

$$\text{vec}(Y) = (B \otimes C) \text{vec}(X).$$

Derivation. Column i of Y is

$$Y(:, i) = \sum_{j=1}^q b_{ij} CX(:, j).$$

The i th block row of $(B \otimes C) \text{vec}(X)$ produces exactly that sum. Stacking the columns proves the identity.

E16: full numerical verification

Use E15's B, C and

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

First,

$$CX = \begin{bmatrix} 2 & 4 \\ 9 & 12 \end{bmatrix}.$$

Then multiply by B^T :

$$Y = \begin{bmatrix} 2 & 4 \\ 9 & 12 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \\ 33 & 12 \end{bmatrix}.$$

Column stacking gives

$$\text{vec}(X) = (1, 3, 2, 4)^T, \quad \text{vec}(Y) = (10, 33, 4, 12)^T.$$

Multiplying the displayed 4×4 Kronecker matrix by $(1, 3, 2, 4)^T$ gives those four entries directly.

Implementation convention. MATLAB's column stacking agrees with this formula. In NumPy, use column-major reshaping with order set to F. Default row stacking changes the formula; it is not merely a cosmetic rearrangement.

Why we should often avoid forming the large matrix

For dense $n \times n$ factors, $B \otimes C$ has n^4 entries. Explicit matrix-vector application takes $O(n^4)$ work. Computing CX and then multiplying by B^T takes two $n \times n$ products, or $O(n^3)$ work, with $O(n^2)$ storage.

This is a precise example of exploiting a representation of an operator instead of materializing every matrix entry.

Added connection: a two-dimensional grid operator

Let $T = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ and

$$U = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

A separable two-dimensional negative-Laplacian stencil has the form

$$\mathcal{L}(U) = TU + UT.$$

Here

$$TU = \begin{bmatrix} -1 & 0 \\ 5 & 6 \end{bmatrix}, \quad UT = \begin{bmatrix} 0 & 3 \\ 2 & 5 \end{bmatrix},$$

so $\mathcal{L}(U) = \begin{bmatrix} -1 & 3 \\ 7 & 11 \end{bmatrix}$. The vec identity gives

$$\text{vec}(\mathcal{L}(U)) = (I \otimes T + T \otimes I) \text{vec}(U).$$

The matrix acting on $(1, 3, 2, 4)^T$ is

$$\begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix}.$$

It returns $(-1, 7, 3, 11)^T$. This connects banded stencils, block matrices, and Kronecker products in one calculation.

7. Hamiltonian and symplectic structure

Slides 50–51; E17. Optional classroom extension. This section illustrates a different meaning of structure: an algebraic identity encoding a geometric property.

For a system with d position and d momentum variables, define

$$J = \begin{bmatrix} 0 & I_d \\ -I_d & 0 \end{bmatrix}.$$

A matrix M is Hamiltonian when

$$M^T J + J M = 0.$$

Write $M = \begin{bmatrix} A & F \\ G & D \end{bmatrix}$. Substitution shows that this identity is equivalent to $D = -A^T$, $F = F^T$, and $G = G^T$.

A step matrix S is symplectic when

$$S^T J S = J.$$

Hamiltonian describes a continuous linear generator; symplectic describes a finite transformation. They are related, but are not interchangeable labels.

E17: verify a shear

In dimension 2,

$$J = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad S = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}.$$

Compute

$$J S = \begin{bmatrix} 0 & 1 \\ -1 & -h \end{bmatrix},$$

and then

$$S^T J S = \begin{bmatrix} 1 & 0 \\ h & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & -h \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} = J.$$

Thus every real h gives a symplectic shear.

For arbitrary u, v , the identity implies

$$(Su)^T J (Sv) = u^T J v.$$

In two dimensions this quantity is the signed area of the parallelogram spanned by the vectors. The transformation preserves that area.

Why explicit Euler can destroy the structure

Take

$$M = \begin{bmatrix} 0 & 1 \\ -4 & 0 \end{bmatrix}.$$

This is the oscillator $q' = p$, $p' = -4q$. Its energy is

$$H(q, p) = 2q^2 + \frac{1}{2}p^2.$$

Explicit Euler uses

$$E = I + hM = \begin{bmatrix} 1 & h \\ -4h & 1 \end{bmatrix}.$$

Direct calculation gives

$$E^T J E = (1 + 4h^2)J.$$

For $h = 1/2$, the area factor is 2; Euler is not symplectic.

Starting from $(q, p) = (1, 0)$ with $h = 1/2$ gives $(q_{\text{new}}, p_{\text{new}}) = (1, -2)$. The energy changes from 2 to 4. The fact that the differential equation is Hamiltonian did not make this numerical update structure-preserving.

As a comparison, first update momentum and then position:

$$p_{\text{new}} = p - 4hq, \quad q_{\text{new}} = q + hp_{\text{new}}.$$

Its matrix is

$$S_h = \begin{bmatrix} 1 - 4h^2 & h \\ -4h & 1 \end{bmatrix},$$

which is a product of symplectic shears. It is symplectic. It does **not** follow that it preserves the exact energy at every step, or that every step size is stable.

Dimension caution. For a 2×2 matrix, $S^T J S = (\det S)J$, so determinant 1 is sufficient. In higher even dimensions, determinant 1 alone is not sufficient for symplecticity.

8. Strassen: reorganizing arithmetic

Slides 52–54; E18–E19. Optional classroom extension. Conventional 2×2 multiplication uses eight scalar multiplications. Strassen replaces them with seven products of sums and differences.

Use the familiar matrices

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

E18: calculate the seven products

Term	Formula	Numerical calculation
M_1	$(a + d)(e + h)$	$(1 + 4)(5 + 8) = 65$
M_2	$(c + d)e$	$(3 + 4)5 = 35$
M_3	$a(f - h)$	$1(6 - 8) = -2$
M_4	$d(g - e)$	$4(7 - 5) = 8$
M_5	$(a + b)h$	$(1 + 2)8 = 24$
M_6	$(c - a)(e + f)$	$(3 - 1)(5 + 6) = 22$
M_7	$(b - d)(g + h)$	$(2 - 4)(7 + 8) = -30$

E19: recombine

$$\begin{aligned} c_{11} &= M_1 + M_4 - M_5 + M_7 = 19, \\ c_{12} &= M_3 + M_5 = 22, \\ c_{21} &= M_2 + M_4 = 43, \\ c_{22} &= M_1 - M_2 + M_3 + M_6 = 50. \end{aligned}$$

Prove one entry rather than simply trusting the recipe:

$$\begin{aligned} M_1 + M_4 - M_5 + M_7 &= (a + d)(e + h) + d(g - e) \\ &\quad - (a + b)h + (b - d)(g + h) \\ &= ae + bg. \end{aligned}$$

Expanding the other combinations gives $af + bh$, $ce + dg$, and $cf + dh$. The proof also works for matrix blocks, because it never reverses the order of a left factor from A and a right factor from B .

Derive the exponent

For $n = 2^\ell$, divide matrices into half-sized blocks:

$$T(n) = 7T(n/2) + cn^2.$$

At recursion level j , there are 7^j subproblems of width $n/2^j$. The addition work at that level is

$$cn^2(7/4)^j.$$

At the leaves there are $7^\ell = n^{\log_2 7}$ scalar products. Summing the geometric series gives

$$T(n) = O(n^{\log_2 7}), \quad \log_2 7 \approx 2.807.$$

At scalar size, the conventional product needs 8 multiplications and 4 additions; this Strassen form uses 7 multiplications and 18 additions. Its benefit is recursive, not a guarantee for a single 2×2 product.

Practical issues. Temporary matrices, memory traffic, the recursion cutoff, padding, and rounding all influence usefulness. State the asymptotic result without claiming that fewer multiplications automatically gives faster execution or better accuracy.

Check. For $n = 8$, recursion to scalars produces $7^3 = 343$ multiplications instead of $8^3 = 512$, but also many additional sums.

9. The DFT is a matrix; the FFT is an algorithm

Slides 55–58; E20–E21. Use zero-based indices in the derivation:

$$\hat{x}_k = \sum_{j=0}^{n-1} x_j \omega_n^{kj}, \quad \omega_n = e^{-2\pi i/n}.$$

The DFT matrix has entries $(F_n)_{kj} = \omega_n^{kj}$. It is generally dense. The FFT evaluates the same linear map by exploiting its algebraic structure.

E20: the four-point DFT

For $n = 4$, $\omega_4 = -i$, so

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$

For $x = (1, 2, 3, 4)^T$,

$$\begin{aligned} \hat{x}_0 &= 1 + 2 + 3 + 4 = 10, \\ \hat{x}_1 &= 1 - 2i - 3 + 4i = -2 + 2i, \\ \hat{x}_2 &= 1 - 2 + 3 - 4 = -2, \\ \hat{x}_3 &= 1 + 2i - 3 - 4i = -2 - 2i. \end{aligned}$$

The zero-frequency coefficient is the sum, not the mean under this normalization. Divide it by n to obtain the mean.

For real inputs, $\hat{x}_{n-k} = \overline{\hat{x}_k}$, with indices taken modulo n . The conjugate pair above provides an arithmetic check.

E21: derive the butterfly

Let $n = 2m$ and separate zero-based even and odd indices:

$$\begin{aligned} \hat{x}_k &= \sum_{j=0}^{m-1} x_{2j} \omega_n^{2kj} + \sum_{j=0}^{m-1} x_{2j+1} \omega_n^{k(2j+1)} \\ &= E_k + \omega_n^k O_k, \end{aligned}$$

where E and O are length- m DFTs because $\omega_n^2 = \omega_m$.

For the output indexed by $k + m$,

$$\omega_n^{k+m} = -\omega_n^k,$$

and the length- m transform outputs repeat periodically. Hence

$$\hat{x}_k = E_k + \omega_n^k O_k, \quad \hat{x}_{k+m} = E_k - \omega_n^k O_k, \quad 0 \leq k < m.$$

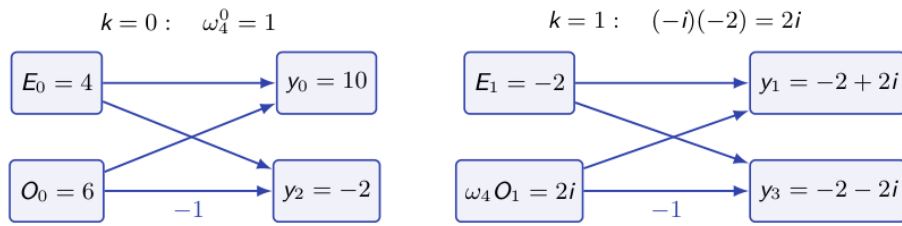
One transformed even/odd pair produces two outputs. This sum-and-difference pattern is the butterfly.

E21: numerical trace

The even-index samples are $(x_0, x_2) = (1, 3)$; the odd-index samples are $(x_1, x_3) = (2, 4)$. Their two-point transforms are

$$E = (1 + 3, 1 - 3) = (4, -2), \quad O = (2 + 4, 2 - 4) = (6, -2).$$

k	E_k	$\omega_4^k O_k$	$\hat{x}_k$	$\hat{x}_{k+2}$
0	4	6	10	-2
1	-2	2i	$-2 + 2i$	$-2 - 2i$



At each right-hand node, sum the incoming contributions.
 Upper output: $E_k + \omega_4^k O_k$; lower output: $E_k - \omega_4^k O_k$.

The two-point transforms of $(1, 3)$ and $(2, 4)$ supply E and O .

Figure 4: Original slide 58: butterfly connections and signs.

In MATLAB, zero-based even samples correspond to positions 1,3,5,... . Thus the slices are $x(1:2:\text{end})$ and $x(2:2:\text{end})$. State the indexing convention before saying “even” and “odd.”

Recursion, inverse, and energy

The recurrence is

$$T(n) = 2T(n/2) + cn,$$

so each recursion level costs cn and there are $\log_2 n$ levels:

$$T(n) = O(n \log n).$$

The simple radix-2 version assumes a power-of-two length. Other lengths require a different decomposition or algorithm.

Fourier columns are orthogonal because a nontrivial geometric sum of roots of unity is zero. Consequently,

$$F_n^* F_n = nI, \quad x = \frac{1}{n} F_n^* \hat{x}.$$

For E20,

$$\|x\|_2^2 = 30, \quad \|\hat{x}\|_2^2 = 100 + 8 + 4 + 8 = 120 = 4(30).$$

The unnormalized DFT scales energy by n . The matrix $F_n/\sqrt{n}$ is unitary and preserves energy.

Check. Applying an FFT twice is not generally the inverse. The inverse changes the exponential sign and includes the appropriate normalization.

10. Sine and cosine transforms: boundaries and normalization

Slides 59–61; E22–E23. These transforms are related to Fourier transforms of specially extended data. Their definitions must specify transform type, endpoints, and scaling.

E22: the DST-I and a discrete eigenvector

For interior indices $j, k = 1, \dots, m-1$, define

$$S_{kj} = \sin\left(\frac{kj\pi}{m}\right).$$

Take $m = 4$, $s = 1/\sqrt{2}$, and $x = (1, 0, -1)^T$:

$$S = \begin{bmatrix} s & 1 & s \\ 1 & 0 & -1 \\ s & -1 & s \end{bmatrix}.$$

Then

$$Sx = \begin{bmatrix} s - s \\ 1 + 1 \\ s - s \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}.$$

Only the second sine coefficient is nonzero.

The size-3 tridiagonal matrix gives

$$Tx = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ -2 \end{bmatrix} = 2x.$$

Thus the input is an eigenvector, not merely a convenient numerical example.

Derive the general sine eigenvalues

For $T = \text{tridiag}(-1, 2, -1)$ of size n , let

$$v_j^{(k)} = \sin(j\theta_k), \quad \theta_k = \frac{k\pi}{n+1}, \quad 1 \leq k \leq n.$$

The endpoint values at $j = 0, n+1$ vanish. Using

$$\sin((j-1)\theta) + \sin((j+1)\theta) = 2 \cos \theta \sin(j\theta)$$

gives

$$Tv^{(k)} = \lambda_k v^{(k)}, \quad \lambda_k = 2 - 2 \cos \theta_k.$$

This explains why sine modes suit homogeneous Dirichlet boundaries. It also links E09's local stencil to a global transform.

Added complete solve using sine coordinates

Solve $Tu = b$ for size 3 and $b = (1, 0, -1)^T$. We know that b is the eigenvector with eigenvalue 2, so $u = b/2$.

The transform route obtains the same answer systematically. Here

$$S^2 = 2I, \quad \Lambda = \text{diag}(2 - \sqrt{2}, 2, 2 + \sqrt{2}).$$

Transform the right-hand side:

$$\hat{b} = Sb = (0, 2, 0)^T.$$

Divide each component by its eigenvalue:

$$\hat{u} = \Lambda^{-1}\hat{b} = (0, 1, 0)^T.$$

Apply the inverse:

$$u = \frac{1}{2}S\hat{u} = (1/2, 0, -1/2)^T.$$

Substitution gives $Tu = b$. For this constant-coefficient, uniform-grid operator, transforms can diagonalize the problem. A variable-coefficient or differently discretized operator need not share these eigenvectors.

E23: the endpoint-half-weight DCT-I

For samples indexed $0, \dots, m$, the slide uses

$$y_k = \frac{x_0}{2} + \sum_{j=1}^{m-1} x_j \cos\left(\frac{kj\pi}{m}\right) + \frac{(-1)^k x_m}{2}, \quad 0 \leq k \leq m.$$

With $m = 2$ and $x = (1, 2, 1)$,

$$y_0 = \frac{1}{2} + 2 + \frac{1}{2} = 3, \quad y_1 = \frac{1}{2} - \frac{1}{2} = 0, \quad y_2 = \frac{1}{2} - 2 + \frac{1}{2} = -1.$$

Form the even extension

$$z = (x_0, x_1, x_2, x_1) = (1, 2, 1, 2).$$

Its four-point DFT is $(6, 0, -2, 0)$. The first three entries equal $2y$.

In general the length- $2m$ extension is

$$(x_0, x_1, \dots, x_m, x_{m-1}, \dots, x_1).$$

Pairing the reflected entries in its DFT gives

$$Z_k = x_0 + (-1)^k x_m + 2 \sum_{j=1}^{m-1} x_j \cos(kj\pi/m) = 2y_k.$$

This derivation explains the factor of two without appealing to a software default.

Convention caution. DCT-I, DCT-II, and orthonormal variants are different definitions. The slide uses DCT-I with half-weighted endpoints. Check both type and normalization before comparing coefficients from different implementations.

11. Haar wavelets: analysis, inverse, and approximation

Slides 62–66; E24–E25. The orthonormal Haar transform replaces each pair by a scaled sum and difference:

$$a = \frac{x_1 + x_2}{\sqrt{2}}, \quad d = \frac{x_1 - x_2}{\sqrt{2}}.$$

The two-by-two analysis matrix is

$$H_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Since $H_2^T H_2 = I$, its inverse is its transpose, which here is also H_2 .

The word “average” in the slides means a normalized sum. It is not the ordinary arithmetic mean. This scaling makes the transform preserve Euclidean energy.

E24: transform all four samples

For $x = (2, 4, 6, 8)^T$, first transform adjacent pairs:

$$a^{(1)} = \left(\frac{2+4}{\sqrt{2}}, \frac{6+8}{\sqrt{2}} \right)^T = (3\sqrt{2}, 7\sqrt{2})^T,$$

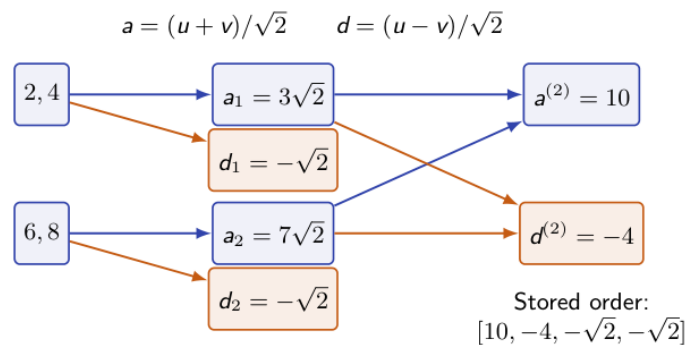
$$d^{(1)} = \left(\frac{2-4}{\sqrt{2}}, \frac{6-8}{\sqrt{2}} \right)^T = (-\sqrt{2}, -\sqrt{2})^T.$$

Transform only the averages:

$$a^{(2)} = \frac{3\sqrt{2} + 7\sqrt{2}}{\sqrt{2}} = 10, \quad d^{(2)} = \frac{3\sqrt{2} - 7\sqrt{2}}{\sqrt{2}} = -4.$$

With coarsest-first coefficient ordering,

$$y = (a^{(2)}, d^{(2)}, d_1^{(1)}, d_2^{(1)})^T = (10, -4, -\sqrt{2}, -\sqrt{2})^T.$$



Blue nodes carry averages to the next level. Orange nodes retain details.

Figure 5: Original slide 64: only averages continue through the decomposition tree.

Write the transform as a matrix

For this coefficient ordering,

$$H_4 = \begin{bmatrix} 1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 & -1/2 \\ 1/\sqrt{2} & -1/\sqrt{2} & 0 & 0 \\ 0 & 0 & 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}.$$

Then $y = H_4 x$. Its rows measure a global average, a difference between halves, and two adjacent-pair differences. Each row has unit norm and distinct rows are orthogonal.

E25: reconstruct step by step

Undo a pair with

$$x_1 = \frac{a+d}{\sqrt{2}}, \quad x_2 = \frac{a-d}{\sqrt{2}}.$$

First recover the level-1 averages:

$$a_1^{(1)} = \frac{10-4}{\sqrt{2}} = 3\sqrt{2}, \quad a_2^{(1)} = \frac{10+4}{\sqrt{2}} = 7\sqrt{2}.$$

Combine each with its stored fine detail:

$$\begin{aligned} x_1 &= (3\sqrt{2} - \sqrt{2})/\sqrt{2} = 2, \\ x_2 &= (3\sqrt{2} + \sqrt{2})/\sqrt{2} = 4, \\ x_3 &= (7\sqrt{2} - \sqrt{2})/\sqrt{2} = 6, \\ x_4 &= (7\sqrt{2} + \sqrt{2})/\sqrt{2} = 8. \end{aligned}$$

Every input sample is recovered exactly in real arithmetic.

Energy and dropping fine details

The input energy is

$$\|x\|_2^2 = 4 + 16 + 36 + 64 = 120.$$

The coefficient energy is

$$\|y\|_2^2 = 100 + 16 + 2 + 2 = 120.$$

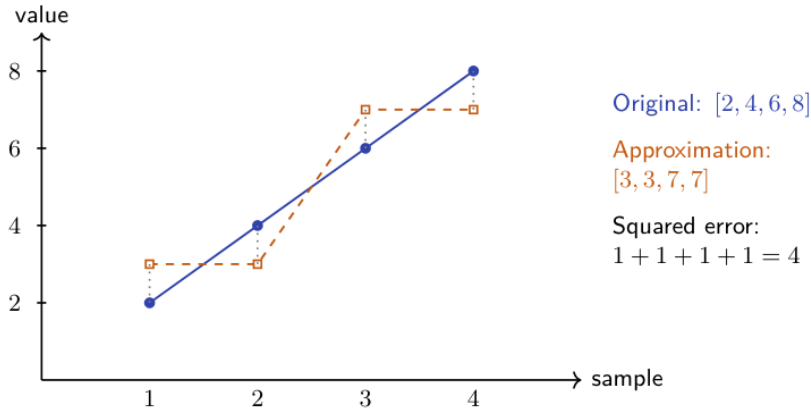
Set the last two coefficients to zero:

$$\tilde{y} = (10, -4, 0, 0)^T.$$

The inverse gives $\tilde{x} = (3, 3, 7, 7)^T$. Thus

$$\|x - \tilde{x}\|_2^2 = (-1)^2 + 1^2 + (-1)^2 + 1^2 = 4.$$

The discarded coefficient energy is also $2 + 2 = 4$.



$(-\sqrt{2})^2 + (-\sqrt{2})^2 = 4$: discarded coefficient energy equals reconstruction error energy.

Figure 6: Original slide 66: removing the fine details produces a piecewise-constant approximation.

This equality follows from orthonormality:

$$\|x - \tilde{x}\|_2^2 = \|H_4(x - \tilde{x})\|_2^2 = \|y - \tilde{y}\|_2^2.$$

Keeping a specified subset of coefficients is the orthogonal projection onto the span of the corresponding basis vectors. It is the best approximation in that fixed subspace.

ML interpretation. A coarse representation can retain global structure while discarding local variations. Discarding details can also remove useful signal; it is not automatically a valid denoising method. The algebra quantifies the approximation error, while an application must decide which information matters.

Complexity and coefficient order

For $n = 2^\ell$, the first level handles n samples, the second $n/2$, and so on:

$$n + n/2 + n/4 + \cdots < 2n.$$

Hence analysis and synthesis cost $O(n)$. Only the averages recurse; the fine details are stored.

With the recursion used in the slides, the output is ordered from the coarsest coefficients to the finest details. A different implementation may store coefficients differently while representing the same transform.

Check. For $x = (c, c, c, c)^T$, the output is $(2c, 0, 0, 0)^T$. All differences vanish and the first coefficient equals the mean times $\sqrt{n}$.

12. Data movement, stride, and blocking

Slides 67–72; E26–E27. Arithmetic count describes only one resource. A processor must obtain input data and store output data. Reusing data in a fast-memory level can be as important as reducing multiplications.

Interpreting the pipeline model on slide 67

Suppose a pipeline has startup cost τ cycles for a vector segment of length v_L , then produces one result per cycle. For n/v_L full segments, its modeled time is

$$\frac{n}{v_L}(\tau + v_L) = n \left(1 + \frac{\tau}{v_L} \right).$$

This explains the form of the slide’s cycle estimate. If the length is not divisible by v_L , a final partial segment needs separate treatment. The model assumes a simple issue rate and does not describe every modern processor.

For $z = x + y$, there are two input streams and one output stream. A simplified nonoverlapping model therefore counts three data streams. Actual arithmetic and data movement may overlap, so adding their modeled times without explaining that assumption can overestimate elapsed time.

Gaxpy versus an outer-product update

Assume 8-byte values, a large $n \times n$ matrix that does not remain in fast memory, and reusable vectors that do fit.

For $y \leftarrow y + Ax$, read A once, read x , and read then write y :

$$\text{ideal traffic} = 8(n^2 + 3n) \text{ bytes.}$$

For $A \leftarrow A + xy^T$, read and write A , and read both vectors:

$$\text{ideal traffic} = 8(2n^2 + 2n) \text{ bytes.}$$

Both operations perform $2n^2$ leading flops, but the outer update moves about twice as much matrix data. Write allocation, repeated loads, and cache behavior can increase actual traffic.

E26: trace addresses

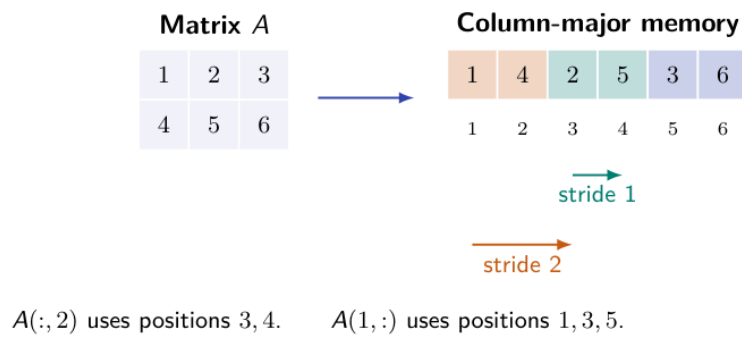
For

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

column-major storage is $[1, 4, 2, 5, 3, 6]$. In one-based positions,

$$\text{position}(i, j) = i + (j - 1)m, \quad m = 2.$$

Column 2 occupies positions 3 and 4, stride 1. Row 1 occupies positions 1,3,5, stride 2.



Position labels use MATLAB indexing. Adjacent entries of a column are adjacent in memory.

Figure 7: Original slide 71: matrix indices and contiguous memory positions.

In row-major storage, one-based position would instead be $(i-1)n+j$. A statement about favorable loop ordering must specify the layout.

For loop order j, k, i , the scalar $B(k, j)$ stays fixed in the inner loop; entries of $A(:, k)$ and $C(:, j)$ are traversed with unit stride in column-major layout.

E27: a cache-capacity calculation

Suppose three $b \times b$ blocks must fit into a 32 KiB budget. Each value uses 8 bytes:

$$3b^2(8) \leq 32768.$$

Therefore

$$b \leq \sqrt{32768/24} \approx 36.95.$$

The largest integer satisfying the simple model is 36. A width of 32 is a convenient feasible candidate:

$$24(32)^2 = 24576 \text{ bytes} = 24 \text{ KiB}.$$

Width 64 needs 98304 bytes, or 96 KiB, and does not fit.

Capacity is only a necessary consideration in this model. Other arrays, associativity conflicts, vector widths, packing, and concurrent threads influence the practical choice. Do not identify 32 as a universally optimal block size.

Check. A hypothetical 24 KiB budget fits three width-32 blocks exactly under this model, leaving no room for other data. A practical implementation may need a smaller width.

13. Parallel work: ownership, balance, and communication

Slides 73–80; E28–E29. A parallel matrix multiplication assigns output blocks to processors. The owner of C_{ij} needs

$$C_{ij} = \sum_k A_{ik} B_{kj}.$$

The arithmetic is independent across output blocks if the required inputs are available and each output has a clear owner.

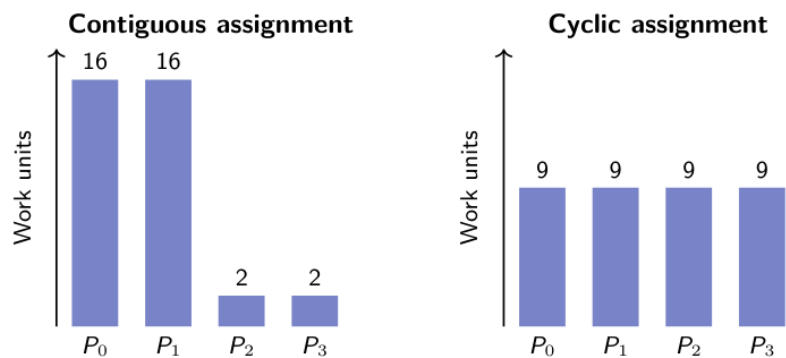
E28: calculate the load balance

Eight tasks have illustrative costs $[8, 8, 8, 8, 1, 1, 1, 1]$. Assign them to four processors.

Assignment	Processor 0	Processor 1	Processor 2	Processor 3
Contiguous pairs	$8 + 8 = 16$	$8 + 8 = 16$	$1 + 1 = 2$	$1 + 1 = 2$
Cyclic, one task at a time	$8 + 1 = 9$	$8 + 1 = 9$	$8 + 1 = 9$	$8 + 1 = 9$

With no communication and equal processor speeds, completion time is the maximum processor load: 16 or 9. The total work is 36, so no assignment can do better than $36/4 = 9$ in this model.

The corresponding speedups over serial work are $36/16 = 2.25$ and $36/9 = 4$. Their efficiencies are 56.25% and 100%.



For the same 36 work units, ideal completion time falls from 16 to 9 when these tasks are assigned cyclically.

Figure 8: Original slide 76: a workload model showing the effect of assignment.

These are synthetic task costs, not a hardware benchmark. If all tasks have equal cost, cyclic assignment gives no arithmetic balancing advantage. It may change locality and communication.

E29: add communication to the timing model

Assume balanced computation, no overlap, and no other overhead:

$$T_p = T_1/p + T_{\text{data}}.$$

For $T_1 = 8$ seconds, $p = 4$, $T_{\text{data}} = 0.5$ seconds,

$$T_4 = 2 + 0.5 = 2.5 \text{ s}, \quad S(4) = 8/2.5 = 3.2, \quad E(4) = S(4)/4 = 0.8.$$

This gives 80% parallel efficiency.

A simple message-time model is

$$T_{\text{message}} = \alpha + \beta\eta,$$

where α is startup latency, η is the word count, and β is time per word.

With $\alpha = 10$ microseconds, $\beta = 0.01$ microseconds/word, and $\eta = 1000$,

$$T_{\text{message}} = 10 + 0.01(1000) = 20 \text{ microseconds.}$$

Sending the same payload as 100 messages of 10 words gives

$$100\{10 + 0.01(10)\} = 1010 \text{ microseconds.}$$

The payload is unchanged; paying latency repeatedly dominates. This motivates aggregation when memory and dependencies permit it.

Shared memory and barriers

Loading one input block at a time uses little local workspace but may reload the same data. Keeping more relevant blocks nearby can reduce redundant transfers, subject to capacity constraints.

A barrier waits until every participating thread reaches it. It can ensure all input initialization finishes before computation starts. It does **not** make simultaneous writes to the same output safe.

If two threads both read $C_{11} = 0$, then each computes a different updated value and writes it, one update can be lost. Assign each output block to one writer, or use a correct reduction/synchronization scheme. A barrier before the updates does not resolve that race.

Check. Ask students to identify which processor owns an output and how it obtains each input block before discussing speedup.

14. Cannon's algorithm: a complete four-processor trace

Slides 81–83; E30. Let processors form a $q \times q$ periodic grid. Processor (i, j) owns output block C_{ij} . Indices in this section start at zero.

Initially it holds A_{ij} and B_{ij} . The initial skew shifts row i of A left by i places and column j of B up by j places. This aligns matching inner indices.

Why the alignment works

After the skew, processor (i, j) holds

$$A_{i, (i+j) \bmod q}, \quad B_{(i+j) \bmod q, j}.$$

After s additional simultaneous left/up shifts, it holds

$$A_{i, k_s}, \quad B_{k_s, j}, \quad k_s = (i + j + s) \bmod q.$$

The inner index matches. As s runs from 0 to $q - 1$, every possible k appears exactly once. Therefore the local accumulated sum is exactly $C_{ij} = \sum_k A_{ik} B_{kj}$.

This invariant is the proof of correctness; a processor diagram alone is not sufficient.

E30: numerical trace with scalar blocks

Use

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad q = 2.$$

After skewing,

$$A_{\text{resident}} = \begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix}, \quad B_{\text{resident}} = \begin{bmatrix} 5 & 8 \\ 7 & 6 \end{bmatrix}.$$

Each displayed position now represents the data resident on that processor, not the original matrix ordering.

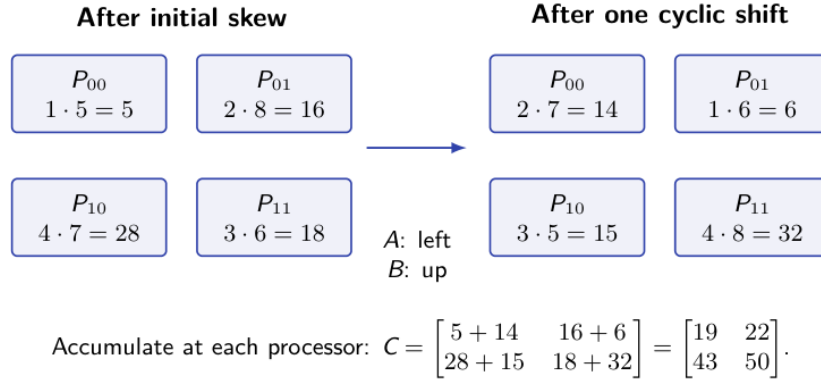
Processor	Round 0	Round 1 after shifting	Final output
(0, 0)	1(5) = 5	2(7) = 14	19
(0, 1)	2(8) = 16	1(6) = 6	22
(1, 0)	4(7) = 28	3(5) = 15	43
(1, 1)	3(6) = 18	4(8) = 32	50

After one left shift of A and up shift of B ,

$$A_{\text{resident}} = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}, \quad B_{\text{resident}} = \begin{bmatrix} 7 & 6 \\ 5 & 8 \end{bmatrix}.$$

The second local products complete

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$



Rows and columns wrap around on the processor torus. Each processor accumulates its own entry of C .

Figure 9: Original slide 83: two rounds on the periodic processor grid.

Work and communication

For an $n \times n$ product on $p = q^2$ processors, each block has width n/q , assuming divisibility. Each processor performs q block products:

$$q \cdot 2(n/q)^3 = \frac{2n^3}{p}$$

leading flops.

Each shift exchanges two blocks, each with $(n/q)^2$ words. Across $O(q)$ stages, the communicated volume per processor scales as

$$O(q(n/q)^2) = O(n^2/\sqrt{p}).$$

Initial alignment, message latency, topology, and overlap affect elapsed time. This volume estimate alone is not a complete timing model.

Check. Without the skew, processor $(0,1)$ would multiply its original a_{01} by b_{01} , whose inner indices do not match. That product is not a valid term of c_{01} .

15. Textbook exercises: assignment

These questions are **paraphrased from verified exercises** in Golub and Van Loan, *Matrix Computations*, 4th edition. The numerical examples and extensions are instructor additions.

Label	Textbook problem	Printed page	Main skill
T1	P1.2.4	21	Rectangular band storage and gaxpy
T2	P1.2.9	21	Symmetric band storage and application
T3	P1.3.7	32	Vec and an outer product
T4	P1.3.5	32	Kronecker product of permutations

Assign T1 and T3 first. T2 and T4 extend the structure arguments.

T1. Rectangular banded gaxpy

Extend banded matrix-vector multiplication to $A \in \mathbb{R}^{m \times n}$. Specify a compact representation and correct index bounds for $y \leftarrow y + Ax$.

Added instance. Use $m = 4$, $n = 3$, $p = q = 1$ and

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 3 & 4 & 5 \\ 0 & 6 & 7 \\ 0 & 0 & 8 \end{bmatrix}, \quad x = (1, 2, -1)^T, \quad y_0 = (1, 1, 1, 1)^T.$$

T2. Symmetric banded gaxpy

Design a compact representation of a symmetric matrix with half-bandwidth p , and an algorithm for $y \leftarrow y + Ax$ that stores only one copy of each symmetric pair.

Added instance. Use E09's size-4 tridiagonal matrix and $x = (1, 2, 3, 4)^T$. Count stored entries and update flops.

T3. Vec of an outer product

For $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$, prove

$$y \otimes x = \text{vec}(xy^T).$$

Added instance. Use $x = (1, 2)^T$, $y = (3, -1, 2)^T$.

T4. Kronecker product of permutations

Show that if P and Q are permutation matrices, then $P \otimes Q$ is also a permutation matrix.

Added instance. Take

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Q = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Apply $P \otimes Q$ to $(1, 2, 3, 4, 5, 6)^T$ by block operations.

16. Textbook exercises: worked solutions

T1 solution: clip both ends to the rectangular dimensions

Use

$$AB(i - j + q + 1, j) = a_{ij}, \quad AB \in \mathbb{R}^{(p+q+1) \times n}.$$

The valid row bounds in column j are

$$\ell_j = \max(1, j - q), \quad u_j = \min(m, j + p).$$

For each j , update all $i = \ell_j, \dots, u_j$. If $\ell_j > u_j$, skip the column.

```
for j = 1, ..., n:
    for i = max(1, j-q), ..., min(m, j+p):
        y[i] += AB[i-j+q+1, j]*x[j]
```

The number of visited entries is

$$N = \sum_{j=1}^n \max(0, u_j - \ell_j + 1).$$

The generic update count is $2N \leq 2n(p + q + 1)$.

For the instance,

$$AB = \begin{bmatrix} * & -1 & 5 \\ 2 & 4 & 7 \\ 3 & 6 & 8 \end{bmatrix}.$$

Its last column contains a_{23}, a_{33}, a_{43} ; this is why the row bound must use m , not n .

The product is

$$Ax = (2 - 2, 3 + 8 - 5, 12 - 7, -8)^T = (0, 6, 5, -8)^T.$$

Therefore

$$y_0 + Ax = (1, 7, 6, -7)^T.$$

The matrix has eight nonzero entries, so the update uses 16 generic flops.

T2 solution: one triangle, paired output updates

Store the lower band in

$$L(r, j) = a_{j+r-1, j}, \quad 1 \leq r \leq p + 1, \quad j + r - 1 \leq n.$$

Row 1 of L contains the diagonal. Each subsequent row holds one subdiagonal.

```

for j = 1,...,n:
    y[j] += L[1,j]*x[j]
    for d = 1,...,min(p,n-j):
        i = j+d
        value = L[d+1,j]
        y[i] += value*x[j]
        y[j] += value*x[i]

```

The number of stored meaningful positions is

$$n + \sum_{d=1}^p (n-d) = n(p+1) - \frac{p(p+1)}{2},$$

assuming $0 \leq p < n$. The rectangular allocated array may include padding.

The update count is

$$2n + 4 \sum_{d=1}^p (n-d) = 2n + 4np - 2p(p+1).$$

This equals twice the number of full-band positions. Symmetric storage removes duplicate values, but both symmetric contributions still enter the product.

For E09,

$$L = \begin{bmatrix} 2 & 2 & 2 & 2 \\ -1 & -1 & -1 & * \end{bmatrix}.$$

There are seven meaningful stored entries. Diagonal updates cost 8 flops, and three off-diagonal pairs cost 12, totaling 20. The output from zero initialization is $(0, 0, 0, 5)^T$.

T3 solution: inspect the columns

The columns of xy^T are $y_1x, \dots, y_nx$. Column stacking produces

$$\text{vec}(xy^T) = \begin{bmatrix} y_1x \\ y_2x \\ \vdots \\ y_nx \end{bmatrix},$$

which is exactly the definition of $y \otimes x$.

For the numerical instance,

$$xy^T = \begin{bmatrix} 3 & -1 & 2 \\ 6 & -2 & 4 \end{bmatrix},$$

and both expressions produce

$$(3, 6, -1, -2, 2, 4)^T.$$

Using row stacking instead would give $(3, -1, 2, 6, -2, 4)^T$, a different order.

T4 solution: prove the one-per-row and one-per-column property

Suppose P has order r and Q has order s . Each block row of $P \otimes Q$ contains one nonzero block, equal to Q , because each row of P contains exactly one 1. Within that block, each scalar row

contains exactly one 1. Thus each row of $P \otimes Q$ has exactly one 1. The same argument applies to columns. All other entries are zero, so it is a permutation matrix.

Orthogonality alone would not prove this: many orthogonal matrices are not permutations. The zero-one pattern is part of the proof.

For the instance,

$$P \otimes Q = \begin{bmatrix} 0 & Q \\ Q & 0 \end{bmatrix}.$$

Partition the input into $z_1 = (1, 2, 3)^T$, $z_2 = (4, 5, 6)^T$. Then

$$(P \otimes Q)z = \begin{bmatrix} Qz_2 \\ Qz_1 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 5 \\ 3 \\ 1 \\ 2 \end{bmatrix}.$$

No explicit 6×6 array is needed to apply it.

17. Additional exercises with solutions

These exercises are **instructor-created** and are separate from the textbook problems above.

P1. A different stencil input

For the size-5 matrix $T = \text{tridiag}(-1, 2, -1)$, calculate Tx for $x = (1, 4, 9, 16, 25)^T$ with zero values beyond the vector.

Solution.

$$Tx = (-2, -2, -2, -2, 34)^T.$$

The interior second differences of i^2 are -2 . At the last row, the implicit boundary value is zero instead of $6^2 = 36$, giving $-16 + 50 = 34$.

P2. A mixed triangular product

If U is upper triangular and L lower triangular, derive the legal range for $(UL)_{ij}$.

Solution. $u_{ik} \neq 0$ requires $k \geq i$; $l_{kj} \neq 0$ requires $k \geq j$. Thus

$$(UL)_{ij} = \sum_{k=\max(i,j)}^n u_{ik}l_{kj}.$$

There is no reason for the output to be triangular.

P3. A four-point FFT check

Compute the unnormalized DFT of $(1, 0, -1, 0)^T$ by even/odd splitting.

Solution. Even samples $(1, -1)$ transform to $(0, 2)$; odd samples $(0, 0)$ transform to $(0, 0)$. The output is $(0, 2, 0, 2)^T$. Its energy 8 equals 4 times the input energy 2.

P4. Haar analysis and a coarse approximation

Transform $x = (1, 1, 3, 3)^T$ using the convention of E24. Keep only the first coefficient and reconstruct.

Solution. First averages are $(\sqrt{2}, 3\sqrt{2})$ and fine details are zero. The final coefficients are $(4, -2, 0, 0)^T$. Keeping only 4 gives $(2, 2, 2, 2)^T$. Squared reconstruction error is 4, equal to the discarded coefficient energy $(-2)^2$.

P5. A cache budget

With an 8 KiB budget and three equal square blocks of 8-byte values, what is the largest integer width permitted by the capacity model?

Solution.

$$24b^2 \leq 8192, \quad b \leq 18.475\dots$$

The largest integer is 18. Width 16 uses 6144 bytes and leaves some room for other data. Neither is automatically the fastest implementation choice.

P6. Scaling with overhead

A serial job takes 12 seconds. On six processors, balanced computation plus a nonoverlapping 1-second overhead is assumed. Calculate speedup and efficiency.

Solution. Parallel time is $12/6 + 1 = 3$ seconds, speedup is 4, and efficiency is $4/6 = 2/3$. The overhead is one-third of parallel elapsed time.

P7. A Cannon index check

On a 3×3 processor grid, list the inner indices visited by processor $(1, 2)$.

Solution. $k_s = (1 + 2 + s) \bmod 3$ gives 0, 1, 2. It accumulates $A_{10}B_{02}$, $A_{11}B_{12}$, and $A_{12}B_{22}$, precisely the three terms of C_{12} .

18. A computational workshop and assessment guide

The complete package includes a small executable Python file, **worked_examples.py**, which verifies representative numerical calculations from both parts. It uses NumPy and implements the teaching algorithms directly where the implementation itself clarifies the mathematics.

Workshop sequence

1. Implement a packed banded product and compare it with an explicitly formed matrix on E11 and textbook exercise T1.
2. Implement the vec identity with column stacking. Intentionally use row stacking once and observe the mismatch.
3. Implement the recursive radix-2 FFT for a power-of-two length. Check the four-point example, inverse normalization, and conjugate symmetry.
4. Implement orthonormal Haar analysis and synthesis. Check exact reconstruction up to rounding, then zero selected details and compare reconstruction-error energy with discarded-coefficient energy.

5. Trace the resident data in Cannon's two-round example. Verify matching inner indices before checking final numbers.

Use small exact integer inputs first, then random floating-point inputs. A numerical tolerance should be scaled to the problem magnitude. A few successful numerical tests illustrate correctness but do not replace the loop-invariant or algebraic proofs.

What students should explain, not merely calculate

Topic	Evidence of understanding
Band structure	Derive bounds and explain missing boundary neighbors.
Symmetry	Show why one stored entry updates two outputs.
Blocks and Kronecker products	State dimensions and preserve multiplication order.
FFT	Derive the twiddle factor and two butterfly outputs.
Haar	Track coefficient order, inverse, and normalization.
Performance	State the data-layout and timing-model assumptions.
Cannon	Identify the resident blocks and prove all inner indices are visited.

Suggested marking emphasis. Give substantial credit for correct setup, assumptions, and intermediate reasoning. Separate an arithmetic slip from an incorrect index range or transform convention.

Final exit discussion

Ask students to complete these sentences:

- A banded matrix saves work because ...
- A Kronecker representation saves storage because ...
- An FFT is fast even though its matrix is dense because ...
- A symmetry can save storage without halving arithmetic because ...
- More processors do not guarantee proportional speedup because ...

Expected ideas: skip structurally impossible interactions; retain factors; reuse smaller transforms; paired entries affect different outputs; communication, imbalance, and synchronization remain.

References and exercise provenance

1. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. Main reading: Sections 1.2–1.6. Textbook exercises P1.2.4 and P1.2.9 are on printed page 21; P1.3.7 and P1.3.5 are on printed page 32. These pages and the edition were checked in the [textbook PDF in a computational-physics teaching repository](#).
2. M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Background on linear maps and orthogonal coordinates. [Author-maintained book site](#).
3. R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2012. Supplementary matrix theory.
4. 王磊, 《矩阵理论与计算》. Chinese-language course reference supplied by the instructor. Edition and publisher details were not supplied. No unverified exercise numbers are attributed to this title.

The slide figures remain the instructor’s supplied illustrations. Textbook statements are paraphrased; all teaching explanations, added examples, and worked solutions are prepared for these notes. The downloaded textbooks are not included in the deliverable package.